

The Tiny Language Definition

Here is the definition for the Tiny language.

The lexicon of the Tiny language is defined as follows:

- Keywords: **WRITE READ IF ELSE RETURN BEGIN END MAIN STRING INT REAL**
- Single-character separators: **; , ()**
- Single-character operators: **+ - * /**
- Multi-character operators: **:= == !=**
- Identifier: An identifier consists of a letter followed by any number of letters or digits. The following are examples of identifiers: x, x2, xx2, x2x, End, END2. Note that **End** is an identifier while **END** is a keyword. The following are not identifiers:
 - IF, WRITE, READ, ... (keywords are not counted as identifiers)
 - 2x (identifier can not start with a digit)
 - Strings in comments are not identifiers.
- Number is a sequence of digits, or a sequence of digits followed by a dot, and followed by digits.

```
Number -> Digits | Digits '.' Digits
Digits -> Digit | Digit Digits
Digit -> '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9'
```

- Comments: string between **/**** and ***/**. Comments can be longer than one line.

The EBNF Grammar

High-level program structures

```
Program -> MethodDecl MethodDecl*
Type -> INT | REAL | STRING
MethodDecl -> Type [MAIN] Id '(' FormalParams ')' Block
FormalParams -> [FormalParam ( ',' FormalParam )* ]
FormalParam -> Type Id
```

Statements

```
Block -> BEGIN Statement+ END

Statement -> Block
           | LocalVarDecl
           | AssignStmt
           | ReturnStmt
           | IfStmt
           | WriteStmt
           | ReadStmt

LocalVarDecl -> Type Id ';' | Type AssignStmt

AssignStmt -> Id := Expression ';'
           | Id := QString ';'

ReturnStmt -> RETURN Expression ';'

IfStmt -> IF '(' BoolExpression ')' Statement
        | IF '(' BoolExpression ')' Statement ELSE Statement

WriteStmt -> WRITE '(' Expression ',' QString ')' ';'
ReadStmt -> READ '(' Id ',' QString ')' ';'

QString is any sequence of characters except double quote itself, enclosed in double quotes.
```

Expressions

```
Expression -> MultiplicativeExpr (( '+' | '-' ) MultiplicativeExpr)*
MultiplicativeExpr -> PrimaryExpr (( '*' | '/' ) PrimaryExpr)*
PrimaryExpr -> Num // Integer or Real numbers
              | Id
              | '(' Expression ')'
              | Id '(' ActualParams ')'
BoolExpression -> Expression '==' Expression
```

```
!Expression '!=' Expression
ActualParams -> [Expression ( ',' Expression)*]
```

Sample program

```
/** this is a comment line in the sample program */
INT f2(INT x, INT y )
BEGIN
    INT z;
    z := x*x - y*y;
    RETURN z;
END
INT MAIN f1()
BEGIN
    INT x;
    READ(x, "A41.input");
    INT y;
    READ(y, "A42.input");
    INT z;
    z := f2(x,y) + f2(y,x);
    WRITE (z, "A4.output");
END
```